

판다스 (Pandas) (3)

2026-1학기

ICT융합학부 컴퓨터공학트랙

김대환

목차

1. 데이터세트 결합: Concat
2. 데이터세트 결합하기: 병합과 조인
3. 집계와 분류
4. 피벗테이블

데이터세트 결합: Concat

데이터세트 결합: Concat (1)

■ 데이터세트 결합

- 데이터 간 겹치는 부분을 제대로 처리하는 복잡한 DB 스타일을 조인하고 병합
- **데이터 랭글링 (Data Wrangling):** 원데이터 (Raw data)를 또 다른 형태로 전환하거나 매핑하는 과정. **데이터 먼징 (Data munging)**이라고도 함

```
import pandas as pd
import numpy as np
```

```
def make_df(cols, ind):
    """Quickly make a DataFrame"""
    data = {c: [str(c) + str(i) for i in ind]
             for c in cols}
    return pd.DataFrame(data, ind)
```

```
# example DataFrame
make_df('ABC', range(3))
```

	A	B	C
0	A0	B0	C0
1	A1	B1	C1
2	A2	B2	C2

데이터세트 결합: Concat (2)

- (복습) NumPy 배열 연결

- np.concatenate 사용하여 두 개 이상의 배열을 하나의 배열로 결합

```
x = [1, 2, 3]
y = [4, 5, 6]
z = [7, 8, 9]
np.concatenate([x, y, z])
```

```
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
x = [[1, 2],
      [3, 4]]
np.concatenate([x, x], axis=1)
```

```
array([[1, 2, 1, 2],
       [3, 4, 3, 4]])
```

데이터셋 결합: Concat (3)

- pd.concat을 이용한 간단 연결

- pd.concat (): 데이터셋 연결

```
pd.concat(objs, axis=0, join='outer', join_axes=None, ignore_index=False, keys=None, levels=None, names=None, verify_integrity=False, copy=True)
```

```
ser1 = pd.Series(['A', 'B', 'C'], index=[1, 2, 3])
ser2 = pd.Series(['D', 'E', 'F'], index=[4, 5, 6])
pd.concat([ser1, ser2])
```

```
1    A
2    B
3    C
4    D
5    E
6    F
dtype: object
```

Series 연결

```
df1 = make_df('AB', [1, 2])
df2 = make_df('AB', [3, 4])
print(df1); print(df2); print(pd.concat([df1, df2]));
```

	A	B
1	A1	B1
2	A2	B2

	A	B
3	A3	B3
4	A4	B4

	A	B
1	A1	B1
2	A2	B2
3	A3	B3
4	A4	B4

```
df3 = make_df('AB', [0, 1])
df4 = make_df('CD', [0, 1])
print(df3); print(df4); print(pd.concat([df3, df4], axis=1))
```

	A	B
0	A0	B0
1	A1	B1

	C	D
0	C0	D0
1	C1	D1

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1

DataFrame 연결

데이터세트 결합: Concat (5)

■ 인덱스 복제

- Pandas에서의 연결은 그 결과가 복제된 인덱스를 가지더라도 인덱스를 그대로 유지

```
x = make_df('AB', [0, 1])
y = make_df('AB', [2, 3])
y.index = x.index # make duplicate indices!
print(x); print(y); print(pd.concat([x,y]))
```

	A	B
0	A0	B0
1	A1	B1

	A	B
0	A2	B2
1	A3	B3

	A	B
0	A0	B0
1	A1	B1
0	A2	B2
1	A3	B3

인덱스가 복제된 채 유지

■ 반복을 에러로 잡아낼 경우

- 인덱스가 겹치지 않는지 검증하고 싶으면 verify_integrity 플래그 지정

```
try:
    pd.concat([x, y], verify_integrity=True)
except ValueError as e:
    print("ValueError:", e)
```

ValueError: Indexes have overlapping values: Index([0, 1], dtype='int64')

데이터세트 결합: Concat (6)

■ 인덱스 복제 (2)

- 인덱스를 무시하고 싶을 경우
 - ignore_index 플래그를 사용

```
print(x); print(y); print(pd.concat([x,y], ignore_index=True))
```

새로운 정수 인덱스 생성

	A	B		A	B		A	B
0	A0	B0	0	A2	B2	0	A0	B0
1	A1	B1	1	A3	B3	1	A1	B1
						2	A2	B2
						3	A3	B3

■ 다중 인덱스 키를 추가

- keys 옵션을 사용. 계층적 인덱스를 가지게 됨

```
print(x); print(y); print(pd.concat([x,y], keys=['x','y']))
```

	A	B		A	B		A	B
0	A0	B0	0	A2	B2	x 0	A0	B0
1	A1	B1	1	A3	B3	1	A1	B1
						y 0	A2	B2
						1	A3	B3

데이터세트 결합: Concat (7)

조인을 이용한 연결

- 다른 열 이름 집합을 가질 경우 (공통적인 열 몇 개만 가질 경우)

```
df5 = make_df('ABC', [1, 2])
df6 = make_df('BCD', [3, 4])
print(df5); print(df6); print(pd.concat([df5, df6]));
```

	A	B	C		B	C	D		A	B	C	D
1	A1	B1	C1	3	B3	C3	D3	1	A1	B1	C1	NaN
2	A2	B2	C2	4	B4	C4	D4	2	A2	B2	C2	NaN
								3	NaN	B3	C3	D3
								4	NaN	B4	C4	D4

채울 값이 없는 항목에는 NaN으로 채움

```
df5 = make_df('ABC', [1, 2])
df6 = make_df('BCD', [3, 4])
print(df5); print(df6); print(pd.concat([df5, df6], join='inner'));
```

join='outer' 합집합, 'inner' 교집합

	A	B	C		B	C	D		B	C
1	A1	B1	C1	3	B3	C3	D3	1	B1	C1
2	A2	B2	C2	4	B4	C4	D4	2	B2	C2
								3	B3	C3
								4	B4	C4

열의 교집합으로 연결

```
df5 = make_df('ABC', [1, 2])
df6 = make_df('BCD', [3, 4])
print(df5); print(df6); print(pd.concat([df5, df6.reindex(columns=df5.columns)]));
```

	A	B	C		B	C	D		A	B	C
1	A1	B1	C1	3	B3	C3	D3	1	A1	B1	C1
2	A2	B2	C2	4	B4	C4	D4	2	A2	B2	C2
								3	NaN	B3	C3
								4	NaN	B4	C4

df5의 열 이름만으로 연결

데이터세트 결합하기: 병합과 조인

데이터세트 결합하기: 병합과 조인 (1)

- 조인 작업의 분류
 - `pd.merge()`: 일대일, 다대일, 다대다 조인 지원
 - 일대일 조인: 열 단위의 연결

```
df1 = pd.DataFrame({'employee': ['Bob', 'Jake', 'Lisa', 'Sue'],  
                    'group': ['Accounting', 'Engineering', 'Engineering', 'HR']})  
df2 = pd.DataFrame({'employee': ['Lisa', 'Bob', 'Jake', 'Sue'],  
                    'hire_date': [2004, 2008, 2012, 2014]})  
print(df1); print(df2)
```

	employee	group		employee	hire_date
0	Bob	Accounting	0	Lisa	2004
1	Jake	Engineering	1	Bob	2008
2	Lisa	Engineering	2	Jake	2012
3	Sue	HR	3	Sue	2014

```
df3 = pd.merge(df1, df2)  
df3
```

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

← 'employee' 열을 가지고 있다는 것을
알고 자동으로 이 열을 키로 사용해 조인

데이터세트 결합하기: 병합과 조인 (2)

■ 조인 작업의 분류 (2)

- 다대일 조인: 두 개의 키 열 중에 열 하나가 중복된 항목을 포함하는 경우의 조인

```
df4 = pd.DataFrame({'group': ['Accounting', 'Engineering', 'HR'],  
                    'supervisor': ['Carly', 'Guido', 'Steve']})  
print(df3); print(df4); print(pd.merge(df3, df4))
```

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

	group	supervisor
0	Accounting	Carly
1	Engineering	Guido
2	HR	Steve

	employee	group	hire_date	supervisor
0	Bob	Accounting	2008	Carly
1	Jake	Engineering	2012	Guido
2	Lisa	Engineering	2004	Guido
3	Sue	HR	2014	Steve

데이터세트 결합하기: 병합과 조인 (3)

■ 조인 작업의 분류 (3)

- 다대다 조인: 왼쪽과 오른쪽 배열의 키 열에 모두 중복 항목이 존재

```
df5 = pd.DataFrame({'group': ['Accounting', 'Accounting',  
                             'Engineering', 'Engineering', 'HR', 'HR'],  
                   'skills': ['math', 'spreadsheets', 'coding', 'linux',  
                             'spreadsheets', 'organization']})  
print(df1); print(df5); print(pd.merge(df1, df5))
```

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR

	group	skills
0	Accounting	math
1	Accounting	spreadsheets
2	Engineering	coding
3	Engineering	linux
4	HR	spreadsheets
5	HR	organization

	employee	group	skills
0	Bob	Accounting	math
1	Bob	Accounting	spreadsheets
2	Jake	Engineering	coding
3	Jake	Engineering	linux
4	Lisa	Engineering	coding
5	Lisa	Engineering	linux
6	Sue	HR	spreadsheets
7	Sue	HR	organization

← 모든 조인 가지수를 조합하여 조인

데이터세트 결합하기: 병합과 조인 (4)

- 병합 키 지정 (1)

- 열 이름이 일치하지 않을 경우의 여러 옵션 지원

- on 키워드

- 열 이름이나 열 이름 리스트를 취하는 on 키워드를 사용해 키 열의 이름을 명시적으로 지정

```
print(df1); print(df2); print(pd.merge(df1, df2, on='employee'))
```

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR

	employee	hire_date
0	Lisa	2004
1	Bob	2008
2	Jake	2012
3	Sue	2014

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

데이터세트 결합하기: 병합과 조인 (5)

- 병합 키 지정 (2)

- left_on가 right_on 키워드

- 다른 열 이름을 가진 두 데이터세트를 병합할 때 사용

```
df3 = pd.DataFrame({'name': ['Bob', 'Jake', 'Lisa', 'Sue'],  
                    'salary': [70000, 80000, 120000, 90000]})  
print(df1); print(df3); print(pd.merge(df1, df3, left_on='employee', right_on='name'))
```

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR

	name	salary
0	Bob	70000
1	Jake	80000
2	Lisa	120000
3	Sue	90000

	employee	group	name	salary
0	Bob	Accounting	Bob	70000
1	Jake	Engineering	Jake	80000
2	Lisa	Engineering	Lisa	120000
3	Sue	HR	Sue	90000

데이터세트 결합하기: 병합과 조인 (6)

- 병합 키 지정 (3)
 - left_index와 right_index 키워드
 - 열을 병합하는 대신 인덱스로 병합

```
df1a = df1.set_index('employee')
df2a = df2.set_index('employee')
print(df1a); print(df2a)
```

	group
employee	
Bob	Accounting
Jake	Engineering
Lisa	Engineering
Sue	HR

	hire_date
employee	
Lisa	2004
Bob	2008
Jake	2012
Sue	2014

```
print(df1a); print(df2a); print(pd.merge(df1a, df2a, left_index=True, right_index=True))
```

	group	hire_date
employee		
Bob	Accounting	
Jake	Engineering	
Lisa	Engineering	
Sue	HR	

	group	hire_date
employee		
Lisa		2004
Bob		2008
Jake		2012
Sue		2014

	group	hire_date
employee		
Bob	Accounting	2008
Jake	Engineering	2012
Lisa	Engineering	2004
Sue	HR	2014

데이터세트 결합하기: 병합과 조인 (7)

■ 병합 키 지정 (4)

- `join()` 메소드: 기본적으로 인덱스 기반으로 병합을 수행

```
print(df1a); print(df2a); print(df1a.join(df2a))
```

group

employee

Bob	Accounting
Jake	Engineering
Lisa	Engineering
Sue	HR

hire_date

employee

Lisa	2004
Bob	2008
Jake	2012
Sue	2014

group hire_date

employee

Bob	Accounting	2008
Jake	Engineering	2012
Lisa	Engineering	2004
Sue	HR	2014

```
print(df1a); print(df3); print(pd.merge(df1a, df3, left_index=True, right_on='name'))
```

group

employee

Bob	Accounting
Jake	Engineering
Lisa	Engineering
Sue	HR

name salary

0	Bob	70000
1	Jake	80000
2	Lisa	120000
3	Sue	90000

group name salary

0	Accounting	Bob	70000
1	Engineering	Jake	80000
2	Engineering	Lisa	120000
3	HR	Sue	90000

데이터세트 결합하기: 병합과 조인 (8)

- 조인을 위한 집합 연산 지정하기

- 어떤 키 열에 등장하는 값이 다른 키 열에서는 등장하지 않는 경우

```
df6 = pd.DataFrame({'name': ['Peter', 'Paul', 'Mary'],  
                    'food': ['fish', 'beans', 'bread']},  
                   columns=['name', 'food'])  
df7 = pd.DataFrame({'name': ['Mary', 'Joseph'],  
                    'drink': ['wine', 'beer']},  
                   columns=['name', 'drink'])  
print(df6); print(df7); print(pd.merge(df6, df7))
```

```
   name  food  
0  Peter  fish  
1   Paul  beans  
2   Mary  bread  
   name drink  
0   Mary  wine  
1  Joseph  beer  
   name food drink  
0   Mary  bread  wine
```

← 공통의 열 값을 가진 항목만 병합
기본적으로 교집합, 내부 조인 (inner join)

```
pd.merge(df6, df7, how='inner')
```

```
   name  food drink  
0   Mary  bread  wine
```

데이터세트 결합하기: 병합과 조인 (9)

■ 조인을 위한 집합 연산 지정하기 (2)

■ how 옵션

- 'outer': 입력 데이터 열의 합집합으로 조인한 결과 반환, 누락된 값은 NA
- 'left': 왼쪽 항목을 기준으로 조인
- 'right': 오른쪽 항목을 기준으로 조인

```
print(df6); print(df7); print(pd.merge(df6, df7, how='outer'))
```

	name	food
0	Peter	fish
1	Paul	beans
2	Mary	bread

	name	drink
0	Mary	wine
1	Joseph	beer

	name	food	drink
0	Joseph	NaN	beer
1	Mary	bread	wine
2	Paul	beans	NaN
3	Peter	fish	NaN

```
print(pd.merge(df6, df7, how='left'))
```

	name	food	drink
0	Peter	fish	NaN
1	Paul	beans	NaN
2	Mary	bread	wine

```
print(pd.merge(df6, df7, how='right'))
```

	name	food	drink
0	Mary	bread	wine
1	Joseph	NaN	beer

데이터세트 결합하기: 병합과 조인 (10)

- 열 이름이 겹치는 경우: `suffixes` 키워드
 - 두 개의 입력에서 충돌하는 열 이름을 가진 경우

```
df8 = pd.DataFrame({'name': ['Bob', 'Jake', 'Lisa', 'Sue'],  
                    'rank': [1, 2, 3, 4]})  
df9 = pd.DataFrame({'name': ['Bob', 'Jake', 'Lisa', 'Sue'],  
                    'rank': [3, 1, 4, 2]})  
print(df8); print(df9); print(pd.merge(df8, df9, on='name'))
```

	name	rank
0	Bob	1
1	Jake	2
2	Lisa	3
3	Sue	4

	name	rank
0	Bob	3
1	Jake	1
2	Lisa	4
3	Sue	2

	name	rank_x	rank_y
0	Bob	1	3
1	Jake	2	1
2	Lisa	3	4
3	Sue	4	2

← 열 고유하게 하기 위하여 자동으로
접미사 `_x` 와 `_y`를 붙임

```
print(df8); print(df9); print(pd.merge(df8, df9, on="name", suffixes=["_L", "_R"]))
```

	name	rank
0	Bob	1
1	Jake	2
2	Lisa	3
3	Sue	4

	name	rank
0	Bob	3
1	Jake	1
2	Lisa	4
3	Sue	2

	name	rank_L	rank_R
0	Bob	1	3
1	Jake	2	1
2	Lisa	3	4
3	Sue	4	2

← 접미사 `_L` 과 `_R`이 붙음

← 접미사 `_L` 과 `_R` 설정

집계와 분류

집계와 분류 (1)

- (데이터셋) 행성데이터

- 다른 별 (외계행성 또는 외행성) 주변에서 발견한 행성에 대한 정보 데이터셋
- Seaborn: 통계적 데이터 시각화 라이브러리

```
import seaborn as sns  
planets = sns.load_dataset('planets')  
planets.shape
```

(1035, 6)

```
planets.head() ← 상위 5개의 데이터를 보여줌
```

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	269.300	7.10	77.40	2006
1	Radial Velocity	1	874.774	2.21	56.95	2008
2	Radial Velocity	1	763.000	2.60	19.84	2011
3	Radial Velocity	1	326.030	19.40	110.62	2007
4	Radial Velocity	1	516.220	10.50	119.47	2009

집계와 분류 (2)

■ Pandas의 간단 집계 연산

■ 최솟값, 최댓값 등등

```
rng = np.random.RandomState(42)
ser = pd.Series(rng.rand(5))
ser
```

```
0    0.374540
1    0.950714
2    0.731994
3    0.598658
4    0.156019
dtype: float64
```

```
ser.sum()
```

```
2.811925491708157
```

```
ser.mean()
```

```
0.5623850983416314
```

Series 집계 연산 예제

```
df = pd.DataFrame({'A': rng.rand(5),
                   'B': rng.rand(5)})
df
```

	A	B
0	0.155995	0.020584
1	0.058084	0.969910
2	0.866176	0.832443
3	0.601115	0.212339
4	0.708073	0.181825

```
df.mean()
```

```
A    0.477888
B    0.443420
dtype: float64
```

```
df.mean(axis='columns')
```

```
0    0.088290
1    0.513997
2    0.849309
3    0.406727
4    0.444949
dtype: float64
```

DataFrame 집계 연산 예제

집계와 분류 (3)

▪ Pandas의 간단 집계 연산 (2)

- **describe()**: 열에 여러 일반적인 집계를 계산하고 반환하는 메서드

```
planets.dropna().describe()
```

	number	orbital_period	mass	distance	year
count	498.00000	498.000000	498.000000	498.000000	498.000000
mean	1.73494	835.778671	2.509320	52.068213	2007.377510
std	1.17572	1469.128259	3.636274	46.596041	4.167284
min	1.00000	1.328300	0.003600	1.350000	1989.000000
25%	1.00000	38.272250	0.212500	24.497500	2005.000000
50%	1.00000	357.000000	1.245000	39.940000	2009.000000
75%	2.00000	999.600000	2.867500	59.332500	2011.000000
max	6.00000	17337.500000	25.000000	354.000000	2014.000000

planets 데이터셋 describe() 결과

Aggregation

Description

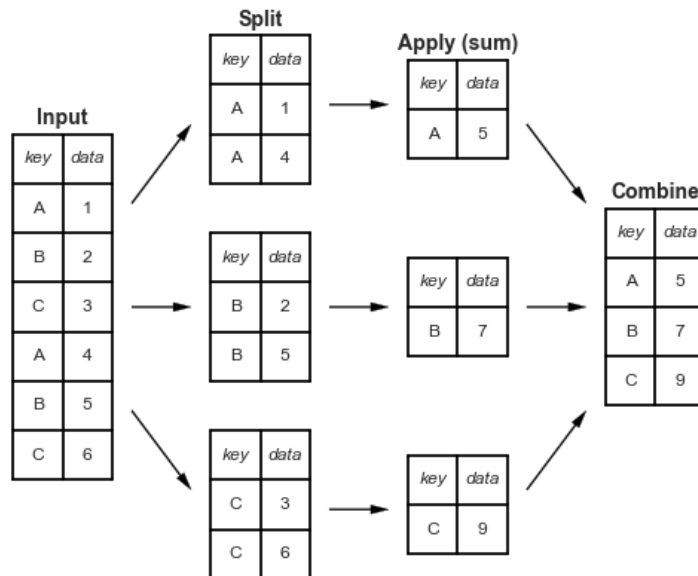
count()	Total number of items
first(), last()	First and last item
mean(), median()	Mean and median
min(), max()	Minimum and maximum
std(), var()	Standard deviation and variance
mad()	Mean absolute deviation
prod()	Product of all items
sum()	Sum of all items

집계 메소드 목록

집계와 분류 (4)

▪ GroupBy: 분할, 적용, 결합 (1)

- 어떤 레이블이나 인덱스를 기준으로 조건부 집계하는 경우 → groupby 연산으로 구현
- 분할 (Split), 적용(Apply), 결합 (Combine)
 - 분할 단계: 지정된 키 값을 기준으로 DataFrame을 나누고 분류하는 단계
 - 적용 단계: 개별 그룹 내에서 일반적으로 집계, 변환, 필터링 같은 함수를 계산
 - 결합 단계: 연산의 결과를 결과 배열에 병합



집계와 분류 (4)

▪ GroupBy: 분할, 적용, 결합 (2)

```
df = pd.DataFrame({'key': ['A', 'B', 'C', 'A', 'B', 'C'],  
                  'data': range(6)}, columns=['key', 'data'])  
df
```

	key	data
0	A	0
1	B	1
2	C	2
3	A	3
4	B	4
5	C	5

```
df.groupby('key') ← 집계 연산 진행 완료
```

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x000001334B62C950>
```

```
df.groupby('key').sum()
```

	data
key	
A	3
B	5
C	7

집계와 분류 (4)

▪ GroupBy: 분할, 적용, 결합 (3)

▪ GroupBy 객체의 중요한 연산들

▪ 집계, 필터, 변환, 적용

▪ 열 인덱싱

▪ GroupBy 객체는 DataFrame과 동일한 방식으로 열 인덱싱을 지원하고 수정된 GroupBy 객체 반환

```
planets.groupby('method')
```

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x000001334B5CECC0>
```

```
planets.groupby('method')['orbital_period']
```

```
<pandas.core.groupby.generic.SeriesGroupBy object at 0x000001334B5CF020>
```

```
planets.groupby('method')['orbital_period'].median()
```

```
method
Astrometry          631.180000
Eclipse Timing Variations  4343.500000
Imaging             27500.000000
Microlensing        3300.000000
Orbital Brightness Modulation    0.342887
Pulsar Timing       66.541900
Pulsation Timing Variations  1170.000000
Radial Velocity     360.200000
Transit             5.714932
Transit Timing Variations  57.011000
Name: orbital_period, dtype: float64
```

집계와 분류 (5)

- GroupBy: 분할, 적용, 결합 (4)

- 그룹 내 반복

- 그룹을 직접 순회할 수 있도록 지원하며, 각 그룹을 Series나 DataFrame으로 반환

```
for (method, group) in planets.groupby('method'):
    print("{0:30s} shape={1}".format(method, group.shape))
```

Astrometry	shape=(2, 6)
Eclipse Timing Variations	shape=(9, 6)
Imaging	shape=(38, 6)
Microlensing	shape=(23, 6)
Orbital Brightness Modulation	shape=(3, 6)
Pulsar Timing	shape=(5, 6)
Pulsation Timing Variations	shape=(1, 6)
Radial Velocity	shape=(553, 6)
Transit	shape=(397, 6)
Transit Timing Variations	shape=(4, 6)

집계와 분류 (6)

■ GroupBy: 분할, 적용, 결합 (5)

■ 디스패치 메소드 (Dispatch method)

- GroupBy 객체가 명시적으로 구현하지 않은 메소드는 DataFrame 객체든 Series 객체든 상관없이 일부 파이썬 클래스 매직을 통해 그 그룹에 전달되고 호출
- (예) DataFrame의 describe() 메소드를 사용해 데이터 집계 연산 수행

```
planets.groupby('method')['year'].describe()
```

	count	mean	std	min	25%	50%	75%	max
method								
Astrometry	2.0	2011.500000	2.121320	2010.0	2010.75	2011.5	2012.25	2013.0
Eclipse Timing Variations	9.0	2010.000000	1.414214	2008.0	2009.00	2010.0	2011.00	2012.0
Imaging	38.0	2009.131579	2.781901	2004.0	2008.00	2009.0	2011.00	2013.0
Microlensing	23.0	2009.782609	2.859697	2004.0	2008.00	2010.0	2012.00	2013.0
Orbital Brightness Modulation	3.0	2011.666667	1.154701	2011.0	2011.00	2011.0	2012.00	2013.0
Pulsar Timing	5.0	1998.400000	8.384510	1992.0	1992.00	1994.0	2003.00	2011.0
Pulsation Timing Variations	1.0	2007.000000	NaN	2007.0	2007.00	2007.0	2007.00	2007.0
Radial Velocity	553.0	2007.518987	4.249052	1989.0	2005.00	2009.0	2011.00	2014.0
Transit	397.0	2011.236776	2.077867	2002.0	2010.00	2012.0	2013.00	2014.0
Transit Timing Variations	4.0	2012.500000	1.290994	2011.0	2011.75	2012.5	2013.25	2014.0

```
planets.groupby('method')['year'].describe().unstack()
```

```
method
count  Astrometry                2.0
       Eclipse Timing Variations  9.0
       Imaging                   38.0
       Microlensing               23.0
       Orbital Brightness Modulation  3.0
       ...
max     Pulsar Timing             2011.0
       Pulsation Timing Variations 2007.0
       Radial Velocity             2014.0
       Transit                   2014.0
       Transit Timing Variations  2014.0
Length: 80, dtype: float64
```

집계와 분류 (7)

■ 집계, 필터, 변환, 적용 (1)

```
rng = np.random.RandomState(0)
df = pd.DataFrame({'key': ['A', 'B', 'C', 'A', 'B', 'C'],
                  'data1': range(6),
                  'data2': rng.randint(0, 10, 6)},
                  columns = ['key', 'data1', 'data2'])
df
```

	key	data1	data2
0	A	0	5
1	B	1	0
2	C	2	3
3	A	3	3
4	B	4	7
5	C	5	9

■ 집계

- **aggregate():** 문자열, 함수, 리스트 등을 취해 한 번에 모든 집계를 계산

```
df.groupby('key').aggregate(['min', np.median, max])
```

key	data1			data2		
	min	median	max	min	median	max
A	0	1.5	3	3	4.0	5
B	1	2.5	4	0	3.5	7
C	2	3.5	5	3	6.0	9

```
df.groupby('key').aggregate({'data1': 'min',
                             'data2': 'max'})
```

key	data1	data2
A	0	5
B	1	7
C	2	9

집계와 분류 (8)

- 집계, 필터, 변환, 적용 (2)

- 필터링

- **filter()**: 그룹이 필터링을 통과하는지 아닌지를 지정하는 부울 값을 반환

```
def filter_func(x):  
    return x['data2'].std() > 4  
print(df); print(df.groupby('key').std()); print(df.groupby('key').filter(filter_func))
```

	key	data1	data2
0	A	0	5
1	B	1	0
2	C	2	3
3	A	3	3
4	B	4	7
5	C	5	9

	key	data1	data2
A	A	2.12132	1.414214
B	B	2.12132	4.949747
C	C	2.12132	4.242641

	key	data1	data2
1	B	1	0
2	C	2	3
4	B	4	7
5	C	5	9

집계와 분류 (9)

- 집계, 필터, 변환, 적용 (3)

- 변환

- **transform()**: 전체 데이터의 변환된 버전을 반환. 입력 값과 같은 형상을 가짐

```
df.groupby('key').transform(lambda x: x - x.mean())
```

	data1	data2
0	-1.5	1.0
1	-1.5	-3.5
2	-1.5	-3.0
3	1.5	-1.0
4	1.5	3.5
5	1.5	3.0

집계와 분류 (10)

- 집계, 필터, 변환, 적용 (4)

- 적용

- **apply():** 임의의 함수를 그룹 결과에 적용할 때 사용
 - DataFrame을 취해 Pandas 객체나 스칼라를 반환

```
def norm_by_data2(x):  
    # x is a DataFrame of group values  
    x['data1'] /= x['data2'].sum()  
    return x  
print(df); print(df.groupby('key').apply(norm_by_data2));
```

	key	data1	data2
0	A	0	5
1	B	1	0
2	C	2	3
3	A	3	3
4	B	4	7
5	C	5	9

	key	data1	data2
A	0	A 0.000000	5
	3	A 0.375000	3
B	1	B 0.142857	0
	4	B 0.571429	7
C	2	C 0.166667	3
	5	C 0.416667	9

집계와 분류 (11)

- 분할 키 지정하기 (1)

- 분할 키 제공하는 리스트, 배열, 시리즈, 인덱스
 - 키는 DataFrame의 길이와 일치하는 길이와 리스트일 수 있음

```
L = [0, 1, 0, 1, 2, 0]
print(df); print(df.groupby(L).sum())
```

	key	data1	data2
0	A	0	5
1	B	1	0
2	C	2	3
3	A	3	3
4	B	4	7
5	C	5	9

	key	data1	data2
0	ACC	7	17
1	BA	4	3
2	B	4	7

```
print(df); print(df.groupby(df['key']).sum())
```

	key	data1	data2
0	A	0	5
1	B	1	0
2	C	2	3
3	A	3	3
4	B	4	7
5	C	5	9

	key	data1	data2
A		3	8
B		5	7
C		7	12

집계와 분류 (12)

- 분할 키 지정하기 (2)

- 인덱스를 그룹에 매핑한 딕셔너리나 시리즈
 - 인덱스 값을 그룹 키에 매핑하는 딕셔너리를 제공하는 것

```
df2 = df.set_index('key')  
mapping = {'A': 'vowel', 'B': 'consonant', 'C': 'consonant'}  
print(df2); print(df2.groupby(mapping).sum())
```

	data1	data2
key		
A	0	5
B	1	0
C	2	3
A	3	3
B	4	7
C	5	9

	data1	data2
key		
consonant	12	19
vowel	3	8

집계와 분류 (13)

- 분할 키 지정하기 (3)

- 파이썬 함수

- 매핑과 유사하게 인덱스 값을 입력해서 그룹을 출력하는 파이썬 함수에 전달

```
print(df2); print(df2.groupby(str.lower).mean())
```

	data1	data2
key		
A	0	5
B	1	0
C	2	3
A	3	3
B	4	7
C	5	9

	data1	data2
key		
a	1.5	4.0
b	2.5	3.5
c	3.5	6.0

- 유효한 키의 리스트

- 앞에서 다룬 모든 키 선택 방식은 다중 인덱스에
서 그룹에 결합 가능

```
df2.groupby([str.lower, mapping]).mean()
```

		data1	data2
key	key		
a	vowel	1.5	4.0
b	consonant	2.5	3.5
c	consonant	3.5	6.0

집계와 분류 (14)

■ 분류 예제

```
decade = 10 * (planets['year'] // 10)
decade = decade.astype(str) + 's'
decade.name = 'decade'
planets.groupby(['method', decade])['number'].sum().unstack().fillna(0)
```

	decade	1980s	1990s	2000s	2010s
	method				
	Astrometry	0.0	0.0	0.0	2.0
	Eclipse Timing Variations	0.0	0.0	5.0	10.0
	Imaging	0.0	0.0	29.0	21.0
	Microlensing	0.0	0.0	12.0	15.0
	Orbital Brightness Modulation	0.0	0.0	0.0	5.0
	Pulsar Timing	0.0	9.0	1.0	1.0
	Pulsation Timing Variations	0.0	0.0	1.0	0.0
	Radial Velocity	1.0	52.0	475.0	424.0
	Transit	0.0	0.0	64.0	712.0
	Transit Timing Variations	0.0	0.0	0.0	9.0

피벗 테이블

피벗 테이블 (1)

■ 피벗 테이블 (Pivot table)

- 입력 값으로 간단한 열 단위의 데이터를 취하고 그 데이터에 대한 다차원 요약을 제공하는 2차원 테이블로 항목을 그룹핑
- 근본적으로 GroupBy 집계의 다차원 버전이라고 생각

■ 데이터셋 (타이타닉 승객 DB)

```
import numpy as np
import pandas as pd
import seaborn as sns
titanic = sns.load_dataset('titanic')
```

```
titanic.head()
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

피벗 테이블 (2)

■ GroupBy 적용 예시

성별에 따른 생존율

```
titanic.groupby('sex')[['survived']].mean()
```

survived	
sex	
female	0.742038
male	0.188908

성별 단위와 좌석 등급으로 그룹을 나누고 → 생존율을 선택 → 평균 집계를 적용 → 결과 그룹 결합 → 계층적 인덱스 분할

```
titanic.groupby(['sex', 'class'])['survived'].aggregate('mean').unstack()
```

class	First	Second	Third
sex			
female	0.968085	0.921053	0.500000
male	0.368852	0.157407	0.135447

→ 너무 복잡한 과정을 거쳐야 함 !!

피벗 테이블 (3)

■ 피벗 테이블 구문

- `pivot_table()` 메서드 사용

```
titanic.pivot_table('survived', index='sex', columns='class')
```

class	First	Second	Third
sex			
female	0.968085	0.921053	0.500000
male	0.368852	0.157407	0.135447

■ 다단계 피벗 테이블

- 피벗 테이블의 그룹핑은 다단계로 여러가지 옵션을 통해 지정할 수 있음
- `pd.cut()`: 값을 고정된 구간으로 나누는 함수
- `pd.qcut()`: 데이터를 같은 개수(분위수)로 나누는 함수

```
age = pd.cut(titanic['age'], [0, 18, 80])
titanic.pivot_table('survived', ['sex', age], 'class')
```

	class	First	Second	Third
sex	age			
female	(0, 18]	0.909091	1.000000	0.511628
	(18, 80]	0.972973	0.900000	0.423729
male	(0, 18]	0.800000	0.600000	0.215686
	(18, 80]	0.375000	0.071429	0.133663

```
fare = pd.qcut(titanic['fare'], 2)
titanic.pivot_table('survived', ['sex', age], [fare, 'class'])
```

	fare	(-0.001, 14.454]			(14.454, 512.329]		
	class	First	Second	Third	First	Second	Third
sex	age						
female	(0, 18]	NaN	1.000000	0.714286	0.909091	1.000000	0.318182
	(18, 80]	NaN	0.880000	0.444444	0.972973	0.914286	0.391304
male	(0, 18]	NaN	0.000000	0.260870	0.800000	0.818182	0.178571
	(18, 80]	0.0	0.098039	0.125000	0.391304	0.030303	0.192308

피벗 테이블 (4)

■ 기타 피벗 테이블 옵션

call signature as of Pandas 0.18

```
DataFrame.pivot_table(data, values=None, index=None, columns=None,  
                        aggfunc='mean', fill_value=None, margins=False,  
                        dropna=True, margins_name='All')
```

■ aggfunc

- 어떤 유형의 집계를 적용할지 제어. 기본으로는 평균이 적용
- 문자열 ('sum', 'mean', 'count', 'min', 'max')이나 집계를 구현하는 함수 (np.sum(), min(), sum()) 으로 지정 가능

```
titanic.pivot_table(index='sex', columns='class',  
                    aggfunc={'survived':sum, 'fare':'mean'})
```

class	fare			survived		
	First	Second	Third	First	Second	Third
sex						
female	106.125798	21.970121	16.118810	91	70	72
male	67.226127	19.741782	12.661633	45	17	47

```
titanic.pivot_table('survived', index='sex', columns='class', margins=True)
```

	class	First	Second	Third	All
sex					
female	0.968085	0.921053	0.500000	0.742038	
male	0.368852	0.157407	0.135447	0.188908	
All	0.629630	0.472826	0.242363	0.383838	

Q & A
